

✂ ДОМЕН 0: ИНСТРУМЕНТЫ (GIT, LINUX, IDE, BASH) — ПОЛНАЯ ДЕТАЛИЗАЦИЯ

СТРУКТУРА КАЖДОЙ ТЕМЫ

Для каждой темы я даю:

- **Что нужно знать** — теория для понимания
 - **Вопросы на собеседовании** (Junior → Senior → Staff)
 - **Практика** — конкретные задания для закрепления
-

РАЗДЕЛ 0.1: GIT (СИСТЕМА КОНТРОЛЯ ВЕРСИЙ)

0.1.1. Базовые команды (Junior Level)

№ 0.1.1.1 — Инициализация репозитория

- **Что нужно знать:** `git init` (создание нового репозитория), `git clone` (копирование удаленного репозитория)
- **Вопросы на собеседовании:** "Как создать новый Git-репозиторий?"
- **Практика:** Создать папку `project`, выполнить `git init`. Создать файл `README.md`, выполнить `git add` и `git commit`

№ 0.1.1.2 — Три состояния файла

- **Что нужно знать:** `modified` (изменен), `staged` (в индексе), `committed` (закоммичен)
- **Вопросы на собеседовании:** "Объясни жизненный цикл файла в Git"
- **Практика:** Изменить файл, посмотреть `git status` (красный) → `git add` (зеленый) → `git commit` (исчез)

№ 0.1.1.3 — Базовые команды

- **Что нужно знать:** `git add` (добавить в индекс), `git commit -m` (зафиксировать), `git status` (текущее состояние), `git log` (история), `git diff` (изменения)
- **Вопросы на собеседовании:** "Что делает команда `git add .`?"
- **Практика:** Создать 3 файла, добавить их по одному, посмотреть историю через `git log --oneline --graph`

№ 0.1.1.4 — Работа с ветками

- **Что нужно знать:** git branch (список/создание), git checkout (переключение), git merge (слияние)
- **Вопросы на собеседовании:** "Как создать новую ветку и переключиться на нее?"
- **Практика:** Создать ветку feature/login, переключиться, сделать коммит, переключиться обратно в main, смиржить

№ 0.1.1.5 — Работа с удаленным репозиторием

- **Что нужно знать:** git remote add origin <url>, git push, git pull, git fetch
- **Вопросы на собеседовании:** "В чем разница между git pull и git fetch?"
- **Практика:** Создать репозиторий на GitHub, запустить локальный проект, сделать изменения на GitHub, выполнить git pull

№ 0.1.1.6 — .gitignore

- **Что нужно знать:** какие файлы игнорировать (бинарники, IDE, зависимости)
- **Вопросы на собеседовании:** "Что такое .gitignore? Приведи пример"
- **Практика:** Создать .gitignore для C++ проекта (игнорировать build/, *.exe, .vscode/)

0.1.2. Продвинутые команды и концепции (Middle Level)

№ 0.1.2.1 — git rebase vs git merge

- **Что нужно знать:** разница, когда что использовать, интерактивный rebase (-i)
- **Вопросы на собеседовании:** "В чем разница между rebase и merge? Когда использовать rebase?"
- **Практика:** Создать ветку с 3 коммитами, выполнить git rebase main, потом попробовать git merge и сравнить историю

№ 0.1.2.2 — git cherry-pick

- **Что нужно знать:** перенос конкретного коммита из одной ветки в другую
- **Вопросы на собеседовании:** "Как перенести один коммит из ветки А в ветку В?"

- **Практика:** Создать ветку feature/bugfix, сделать коммит, переключиться на main и выполнить git cherry-pick <hash>

№ 0.1.2.3 — git stash

- **Что нужно знать:** временное сохранение незакоммиченных изменений (push, pop, apply, list)
- **Вопросы на собеседовании:** "Что делать, если нужно переключиться на другую ветку, а изменения еще не закоммичены?"
- **Практика:** Изменить файл, выполнить git stash, переключить ветку, вернуться, выполнить git stash pop

№ 0.1.2.4 — git reset vs git revert

- **Что нужно знать:** reset (откат истории, опасен) vs revert (новый коммит, безопасен)
- **Вопросы на собеседовании:** "Как отменить последний коммит? В чем разница между reset и revert?"
- **Практика:** Сделать 3 коммита, отменить последний через git reset --hard HEAD~1, потом восстановить через git reflog

№ 0.1.2.5 — git reflog

- **Что нужно знать:** восстановление потерянных коммитов (важно для Senior)
- **Вопросы на собеседовании:** "Как восстановить коммит, который был удален через git reset --hard?"
- **Практика:** Удалить коммит, найти его хеш в reflog, выполнить git reset --hard <hash>

№ 0.1.2.6 — Разрешение конфликтов

- **Что нужно знать:** ручное редактирование файлов, git mergetool, стратегии слияния (ours, theirs)
- **Вопросы на собеседовании:** "Как разрешить конфликт при merge?"
- **Практика:** Создать две ветки, изменить одну и ту же строку в файле, выполнить git merge, вручную разрешить конфликт

№ 0.1.2.7 — git bisect

- **Что нужно знать:** бинарный поиск коммита, который ввел баг
- **Вопросы на собеседовании:** "Как найти коммит, который сломал сборку, если известно, что 3 дня назад все работало?"

- **Практика:** Создать историю с 10 коммитами, пометить один как "баг", выполнить git bisect для поиска

№ 0.1.2.8 — Работа с submodules

- **Что нужно знать:** добавление внешнего репозитория как подмодуля
 - **Вопросы на собеседовании:** "Что такое submodule и зачем он нужен?"
 - **Практика:** Добавить внешний репозиторий как submodule, обновить его, закоммитить изменения
-

0.1.3. Экспертный уровень (Senior / Staff Level)

№ 0.1.3.1 — Внутреннее устройство Git

- **Что нужно знать:** объекты (blob, tree, commit, tag), как хранятся данные (папка .git/objects), хеши SHA-1
- **Вопросы на собеседовании:** "Как Git хранит файлы внутри? Что такое blob, tree, commit?"
- **Практика:** Зайти в .git/objects, найти файл коммита, выполнить git cat-file -p <hash> и посмотреть структуру

№ 0.1.3.2 — Git Hooks

- **Что нужно знать:** pre-commit, commit-msg, pre-push (автоматизация проверок)
- **Вопросы на собеседовании:** "Как автоматически запускать линтер перед каждым коммитом?"
- **Практика:** Написать pre-commit хук, который проверяет форматирование кода (clang-format)

№ 0.1.3.3 — Управление историей

- **Что нужно знать:** git filter-branch, git rebase --interactive (сжатие коммитов, изменение сообщений)
- **Вопросы на собеседовании:** "Как объединить несколько коммитов в один (squash)?"
- **Практика:** Взять ветку с 5 коммитами, выполнить git rebase -i HEAD~5, сделать squash первых 3 коммитов

№ 0.1.3.4 — Стратегии ветвления

- **Что нужно знать:** Git Flow vs GitHub Flow vs Trunk-Based Development

- **Вопросы на собеседовании:** "Какую стратегию ветвления ты используешь и почему?"
- **Практика:** Сравнить Git Flow (сложный), GitHub Flow (простой), Trunk-Based (для CI/CD) — написать эссе

№ 0.1.3.5 — Управление большими файлами

- **Что нужно знать:** Git LFS (Large File Storage)
- **Вопросы на собеседовании:** "Как работать с бинарными файлами (архивы, образы) в Git?"
- **Практика:** Настроить Git LFS для отслеживания .psd и .zip файлов

№ 0.1.3.6 — Безопасность

- **Что нужно знать:** подпись коммитов (GPG), как защитить ветку main от прямых пушей
- **Вопросы на собеседовании:** "Как защитить ветку main от случайного push-a?"
- **Практика:** Настроить branch protection rule в GitHub: require pull request, require status checks

№ 0.1.3.7 — Оптимизация производительности

- **Что нужно знать:** git gc (сборка мусора), git pack-objects, git repack
- **Вопросы на собеседовании:** "Почему Git может работать медленно в большом проекте? Что делать?"
- **Практика:** Запустить git gc --aggressive в репозитории с большой историей

№ 0.1.3.8 — Git internals (для архитекторов)

- **Что нужно знать:** как работает git push (pack-файлы, дельта-компрессия)
- **Вопросы на собеседовании:** "Как Git передает изменения при git push?"
- **Практика:** Изучить статью "Git Internals" и написать конспект

0.1.4. Практические задания (сквозные)

№ 0.1.4.1

- **Задание:** Написать скрипт на Bash, который автоматически создает новую ветку, делает коммит и создает pull request через GitHub CLI (gh)
- **Что проверяет:** Знание Git + Bash + GitHub API

№ 0.1.4.2

- **Задание:** Найти коммит, который сломал сборку, используя git bisect
- **Что проверяет:** Умение пользоваться бинарным поиском

№ 0.1.4.3

- **Задание:** Перенести историю из SVN/CVS в Git (изучить git svn)
- **Что проверяет:** Понимание миграции систем контроля версий

№ 0.1.4.4

- **Задание:** Настроить Git hooks для автоматического запуска тестов перед push
- **Что проверяет:** Знание автоматизации

№ 0.1.4.5

- **Задание:** Создать собственный Git-сервер через SSH
- **Что проверяет:** Понимание сетевой работы Git

РАЗДЕЛ 0.2: LINUX (ОПЕРАЦИОННАЯ СИСТЕМА)

0.2.1. Основы работы в терминале (Junior Level)

№ 0.2.1.1 — Навигация

- **Что нужно знать:** pwd (текущая директория), ls (список файлов), cd (переход), mkdir, rmdir
- **Вопросы на собеседовании:** "Как вывести текущую директорию?"
- **Практика:** Изучить структуру файловой системы Linux: /, /etc, /var, /usr, /home, /tmp

№ 0.2.1.2 — Работа с файлами

- **Что нужно знать:** touch (создание), cp (копирование), mv (перемещение/переименование), rm (удаление)
- **Вопросы на собеседовании:** "Как удалить папку со всем содержимым?"

- **Практика:** Создать структуру папок `project/src`, `project/build`, `project/docs` и заполнить файлами

№ 0.2.1.3 — Права доступа

- **Что нужно знать:** `chmod` (изменение прав), `chown` (смена владельца), `umask`
- **Вопросы на собеседовании:** "Что означают права `rwxtg-xr--?`"
- **Практика:** Создать файл, изменить права на `755`, объяснить каждый бит

№ 0.2.1.4 — Просмотр файлов

- **Что нужно знать:** `cat` (вывод), `less/more` (постраничный просмотр), `head/tail` (начало/конец)
- **Вопросы на собеседовании:** "Как посмотреть последние 20 строк лог-файла?"
- **Практика:** Открыть лог-файл, использовать `tail -f` для просмотра в реальном времени

№ 0.2.1.5 — Переменные окружения

- **Что нужно знать:** `export`, `env`, `echo $PATH`, `~/.bashrc`
- **Вопросы на собеседовании:** "Как добавить директорию в `PATH`?"
- **Практика:** Добавить `/usr/local/bin` в `PATH` в `.bashrc`

№ 0.2.1.6 — Процессы

- **Что нужно знать:** `ps` (список), `top/htop` (мониторинг), `kill` (завершение), `kill -9`, `nice` (приоритет)
- **Вопросы на собеседовании:** "Как завершить процесс по `PID`?"
- **Практика:** Запустить бесконечный цикл в терминале, найти его `PID` через `ps aux`, завершить через `kill`

0.2.2. Работа с процессами и сетью (Middle Level)

№ 0.2.2.1 — Планировщик

- **Что нужно знать:** `cron` (задачи по расписанию), `crontab` (редактирование)

- **Вопросы на собеседовании:** "Как настроить ежедневный бэкап в 3 часа ночи?"
- **Практика:** Написать cron-задачу: `0 3 * * * /home/user/backup.sh`

№ 0.2.2.2 — Управление пакетами

- **Что нужно знать:** apt (Debian/Ubuntu), yum/dnf (RHEL/Fedora)
- **Вопросы на собеседовании:** "Как установить пакет в Ubuntu?"
- **Практика:** Установить build-essential, cmake, gdb через apt

№ 0.2.2.3 — Системные вызовы

- **Что нужно знать:** strace (трассировка системных вызовов процесса)
- **Вопросы на собеседовании:** "Как понять, с какими файлами работает программа?"
- **Практика:** Запустить `strace -e trace=file ls` и посмотреть, какие файлы открывает ls

№ 0.2.2.4 — Сеть

- **Что нужно знать:** netstat (открытые порты), ss (альтернатива netstat), tcpdump (анализ трафика)
- **Вопросы на собеседовании:** "Как узнать, какой процесс слушает порт 8080?"
- **Практика:** Выполнить `ss -tulpn | grep 8080`, найти PID процесса

№ 0.2.2.5 — Мониторинг

- **Что нужно знать:** vmstat (память/процессы), iostat (диски), free (память), df/du (дисковое пространство)
- **Вопросы на собеседовании:** "Как проверить, сколько свободной оперативной памяти?"
- **Практика:** Выполнить `free -h`, объяснить, что такое buffers/cache

№ 0.2.2.6 — systemd

- **Что нужно знать:** создание сервисов (unit files), systemctl start/stop/status/enable
- **Вопросы на собеседовании:** "Как сделать, чтобы твоё приложение запускалось автоматически при старте системы?"

- **Практика:** Создать `.service` файл для своего микросервиса, выполнить `systemctl enable`
-

0.2.3. Экспертный уровень (Senior / Staff Level)

№ 0.2.3.1 — Файловые системы

- **Что нужно знать:** `ext4`, `XFS`, `Btrfs`, `ZFS` (особенности, журналирование), `inode`
- **Вопросы на собеседовании:** "Что такое `inode`?"
- **Практика:** Создать файл, выполнить `ls -li` (показать `inode`), объяснить, как работает поиск файла

№ 0.2.3.2 — Управление памятью

- **Что нужно знать:** виртуальная память (`swap`), `mmap` (отображение файлов в память)
- **Вопросы на собеседовании:** "Как работает `swap`? Когда он используется?"
- **Практика:** Создать `swap`-файл, включить его, проверить `swapon -s`

№ 0.2.3.3 — I/O Schedulers

- **Что нужно знать:** `CFQ`, `Deadline`, `Noop` (как планируются операции чтения/записи)
- **Вопросы на собеседовании:** "Как увеличить производительность диска в базе данных?"
- **Практика:** Изучить, как изменить `scheduler` для диска (`/sys/block/sdX/queue/scheduler`)

№ 0.2.3.4 — Cgroups и Namespaces

- **Что нужно знать:** механизмы изоляции (основа `Docker`)
- **Вопросы на собеседовании:** "Как `Docker` изолирует контейнеры?"
- **Практика:** Создать процесс в новой `network namespace`, изучить `lsns`

№ 0.2.3.5 — SELinux / AppArmor

- **Что нужно знать:** системы мандатного контроля доступа
- **Вопросы на собеседовании:** "Что такое `SELinux` и зачем он нужен?"

- **Практика:** Включить SELinux в permissive mode, изучить логи (/var/log/audit/audit.log)

№ 0.2.3.6 — Performance Tuning

- **Что нужно знать:** sysctl (настройка ядра), ulimit (лимиты процессов)
- **Вопросы на собеседовании:** "Как увеличить лимит открытых файлов для процесса?"
- **Практика:** Изменить ulimit -n 65535, объяснить, где это может пригодиться

№ 0.2.3.7 — Kernel debugging

- **Что нужно знать:** /proc, /sys, dmesg (сообщения ядра)
- **Вопросы на собеседовании:** "Как узнать, почему сервер перезагрузился?"
- **Практика:** Проанализировать dmesg | tail -50 и найти причину паники ядра

№ 0.2.3.8 — Анализ производительности

- **Что нужно знать:** perf (профилирование CPU), ftrace (трассировка функций ядра)
- **Вопросы на собеседовании:** "Как найти узкое место в производительности приложения?"
- **Практика:** Запустить perf record для программы, проанализировать perf report

0.2.4. Практические задания (сквозные)

№ 0.2.4.1

- **Задание:** Написать Bash-скрипт, который находит все .log файлы старше 30 дней и архивирует их в .tar.gz
- **Что проверяет:** Знание Bash + работа с файлами

№ 0.2.4.2

- **Задание:** Настроить системный мониторинг: Prometheus + node_exporter на Linux сервере
- **Что проверяет:** Знание DevOps практик

№ 0.2.4.3

- **Задание:** Написать скрипт, который парсит логи nginx и выводит топ-10 IP-адресов по количеству запросов
- **Что проверяет:** Умение работать с awk, sort, uniq

№ 0.2.4.4

- **Задание:** Создать systemd-сервис для приложения на C++ с автоматическим перезапуском при падении
- **Что проверяет:** Знание управления сервисами

№ 0.2.4.5

- **Задание:** Настроить chroot окружение для запуска изолированного приложения
- **Что проверяет:** Понимание изоляции процессов

РАЗДЕЛ 0.3: IDE И РАЗРАБОТКА

0.3.1. Базовые навыки (Junior Level)

№ 0.3.1.1 — VS Code

- **Что нужно знать:** базовые функции, терминал внутри IDE, расширения (C++, C#, GitLens)
- **Вопросы на собеседовании:** "Какие расширения ты используешь для разработки на C++?"
- **Практика:** Установить VS Code, настроить C++ окружение (CMake Tools, C++ IntelliSense)

№ 0.3.1.2 — CLion

- **Что нужно знать:** настройка компилятора (GCC/Clang), CMake интеграция, отладчик GDB
- **Вопросы на собеседовании:** "Как настроить сборку C++ проекта в CLion?"
- **Практика:** Создать CMake проект в CLion, добавить несколько библиотек, настроить дебаг конфигурации

№ 0.3.1.3 — Rider / Visual Studio

- **Что нужно знать:** работа с решениями (sln), проектами (csproj), NuGet пакетами
- **Вопросы на собеседовании:** "Как добавить библиотеку в C# проект?"

- **Практика:** Создать консольное приложение на C#, добавить NuGet пакет (Newtonsoft.Json)

№ 0.3.1.4 — Отладка (база)

- **Что нужно знать:** установка точек останова (breakpoints), пошаговое выполнение (step into/over/out), просмотр переменных (watch)
- **Вопросы на собеседовании:** "Что такое breakpoint и как пользоваться watch?"
- **Практика:** Написать программу с багом, запустить дебаггер, пройти по шагам, найти ошибку

№ 0.3.1.5 — Форматирование кода

- **Что нужно знать:** clang-format (C++), dotnet format (C#), EditorConfig
 - **Вопросы на собеседовании:** "Как автоматически форматировать код в проекте?"
 - **Практика:** Создать .clang-format файл, настроить стиль (Google/LLVM), применить к проекту
-

0.3.2. Продвинутые навыки (Middle Level)

№ 0.3.2.1 — Удаленная разработка

- **Что нужно знать:** VS Code Remote SSH, Remote Containers (разработка внутри Docker)
- **Вопросы на собеседовании:** "Как разрабатывать код, который выполняется на удаленном сервере?"
- **Практика:** Настроить VS Code Remote SSH для разработки на удаленной VM

№ 0.3.2.2 — Продвинутая отладка

- **Что нужно знать:** условные точки останова, точки данных (data breakpoint), трассировка стека, core dump анализ
- **Вопросы на собеседовании:** "Как отлаживать программу, которая падает только в продакшене?"
- **Практика:** Сгенерировать core dump, загрузить его в GDB, проанализировать стек вызовов

№ 0.3.2.3 — Профилирование

- **Что нужно знать:** perf (Linux), dotnet-trace (C#), Intel VTune
- **Вопросы на собеседовании:** "Как найти функцию, которая потребляет больше всего CPU?"
- **Практика:** Запустить профайлер на своей программе, найти узкое место, оптимизировать

№ 0.3.2.4 — Линтеры и статические анализаторы

- **Что нужно знать:** clang-tidy (C++), SonarQube, ESLint (JS)
- **Вопросы на собеседовании:** "Что такое статический анализ кода? Приведи примеры"
- **Практика:** Настроить clang-tidy для C++ проекта и исправить все warnings

№ 0.3.2.5 — Интеграция с Git

- **Что нужно знать:** визуализация истории, управление ветками через IDE, diff/merge инструменты
- **Вопросы на собеседовании:** "Как в IDE разрешить конфликт при merge?"
- **Практика:** Создать конфликт, открыть его в IDE, использовать встроенный merge tool

0.3.3. Экспертный уровень (Senior / Staff Level)

№ 0.3.3.1 — Настройка IDE для команды

- **Что нужно знать:** общие конфигурации (.editorconfig, .vscode/extensions.json), шаблоны кода
- **Вопросы на собеседовании:** "Как настроить единый стиль кода для всей команды?"
- **Практика:** Создать .editorconfig для C++ и C#, добавить в репозиторий, настроить автоматическое применение

№ 0.3.3.2 — Кастомизация

- **Что нужно знать:** создание своих сниппетов, шаблонов файлов (file templates), макросы
- **Вопросы на собеседовании:** "Как автоматизировать создание boilerplate кода?"

- **Практика:** Создать сниппет для класса C++ с конструктором и деструктором

№ 0.3.3.3 — Интеграция с CI/CD

- **Что нужно знать:** настройка плагинов для CI (GitHub Actions, GitLab CI)
- **Вопросы на собеседовании:** "Как запустить тесты из IDE и увидеть результат в CI?"
- **Практика:** Настроить пайплайн GitLab CI, который запускает тесты при push'e

№ 0.3.3.4 — Анализ производительности IDE

- **Что нужно знать:** оптимизация работы с большими проектами (индексация, кэширование)
- **Вопросы на собеседовании:** "Что делать, если IDE тормозит в большом проекте?"
- **Практика:** Изучить настройки памяти для IntelliJ (CLion/Rider): -Xmx параметры

№ 0.3.3.5 — Плагины для безопасности

- **Что нужно знать:** Snyk, Dependabot (анализ уязвимостей зависимостей)
- **Вопросы на собеседовании:** "Как автоматически проверять зависимости на уязвимости?"
- **Практика:** Настроить Dependabot в GitHub для автоматического обновления библиотек

РАЗДЕЛ 0.4: BASH / SHELL (АВТОМАТИЗАЦИЯ)

0.4.1. Основы Bash (Junior Level)

№ 0.4.1.1 — Переменные

- **Что нужно знать:** объявление (VAR="value"), использование (\$VAR), кавычки (одинарные vs двойные)
- **Вопросы на собеседовании:** "В чем разница между '\$VAR' и '\$VAR'?"
- **Практика:** Написать скрипт, который выводит приветствие с именем пользователя

№ 0.4.1.2 — Условные операторы

- **Что нужно знать:** if, elif, else, case, test ([]), [[]]
- **Вопросы на собеседовании:** "Как проверить существование файла в Bash?"
- **Практика:** Написать скрипт, который проверяет наличие файла config.ini и создает его, если нет

№ 0.4.1.3 — Циклы

- **Что нужно знать:** for, while, until
- **Вопросы на собеседовании:** "Как перебрать все файлы в папке?"
- **Практика:** Написать скрипт, который переименовывает все .txt файлы в .bak

№ 0.4.1.4 — Функции

- **Что нужно знать:** объявление (function_name() {}), параметры (\$1, \$2, @\$), возврат (return, echo)
- **Вопросы на собеседовании:** "Как передать аргументы в Bash функцию?"
- **Практика:** Написать функцию log_error, которая записывает сообщение в файл ошибок

№ 0.4.1.5 — Ввод-вывод

- **Что нужно знать:** перенаправление (> для перезаписи, >> для добавления), конвейер (|), read
- **Вопросы на собеседовании:** "Что делает grep error log.txt | wc -l?"
- **Практика:** Написать скрипт, который читает ввод от пользователя и обрабатывает его

0.4.2. Продвинутые скрипты (Middle Level)

№ 0.4.2.1 — Обработка ошибок

- **Что нужно знать:** set -e (выход при ошибке), set -u (неопределенные переменные), trap (перехват сигналов)
- **Вопросы на собеседовании:** "Как сделать скрипт безопасным и не продолжать выполнение при ошибке?"

- **Практика:** Написать скрипт с `set -e`, добавить `trap` для очистки временных файлов при выходе

№ 0.4.2.2 — Работа с текстом

- **Что нужно знать:** `grep` (поиск), `awk` (обработка колонок), `sed` (замена текста), `cut`, `sort`, `uniq`
- **Вопросы на собеседовании:** "Как вывести только второй столбец из CSV файла?"
- **Практика:** Написать скрипт, который парсит лог, выводит топ-5 IP-адресов по количеству запросов

№ 0.4.2.3 — Работа с файлами и директориями

- **Что нужно знать:** `find` (поиск файлов), `xargs` (передача аргументов), `tar/gzip` (архивация)
- **Вопросы на собеседовании:** "Как найти все файлы `.log` за последние 7 дней и удалить их?"
- **Практика:** Написать скрипт для архивации логов старше 30 дней и удаления оригиналов

№ 0.4.2.4 — Регулярные выражения

- **Что нужно знать:** `basic` vs `extended` (`grep -E`), группы, квантификаторы
- **Вопросы на собеседовании:** "Как найти все email-адреса в текстовом файле?"
- **Практика:** Написать скрипт, который извлекает все email-адреса из файла

№ 0.4.2.5 — Работа с сетью

- **Что нужно знать:** `curl` (запросы), `wget` (скачивание), `nc` (`netcat`)
- **Вопросы на собеседовании:** "Как отправить POST-запрос из Bash-скрипта?"
- **Практика:** Написать скрипт, который отправляет HTTP-запрос к API и проверяет статус ответа

0.4.3. Экспертный уровень (Senior / Staff Level)

№ 0.4.3.1 — Автоматизация деплоя

- **Что нужно знать:** скрипты для развертывания приложений, управление окружениями (dev/stage/prod)
- **Вопросы на собеседовании:** "Как автоматизировать деплой микросервиса в Kubernetes?"
- **Практика:** Написать скрипт, который собирает образ, пушит в registry, обновляет Helm-чарт

№ 0.4.3.2 — CI/CD пайплайны

- **Что нужно знать:** Bash-скрипты для GitLab CI / GitHub Actions
- **Вопросы на собеседовании:** "Как написать скрипт, который собирает C++ проект и запускает тесты?"
- **Практика:** Написать пайплайн для сборки C++ проекта с CMake и запуска тестов

№ 0.4.3.3 — Парсинг JSON/YAML

- **Что нужно знать:** jq (JSON), уq (YAML)
- **Вопросы на собеседовании:** "Как извлечь значение из JSON-ответа API?"
- **Практика:** Написать скрипт, который получает курс валют через API и выводит только значение

№ 0.4.3.4 — Диагностика и отладка скриптов

- **Что нужно знать:** set -x (трассировка), анализ кода выхода (\$?), логирование
- **Вопросы на собеседовании:** "Как отлаживать Bash-скрипт, который падает без понятной ошибки?"
- **Практика:** Запустить скрипт с bash -x script.sh, проанализировать вывод

№ 0.4.3.5 — Безопасность скриптов

- **Что нужно знать:** избегание eval, правильная обработка аргументов (с пробелами), использование getopts
 - **Вопросы на собеседовании:** "Как написать скрипт, который принимает параметры с ключами (-f filename)?"
 - **Практика:** Написать скрипт с использованием getopts для обработки параметров -i и -o
-

🔗 СВОДНЫЙ ЧЕК-ЛИСТ ДЛЯ ПОДГОТОВКИ К СОБЕСЕДОВАНИЮ

Junior Level (должен знать 100%)

Git: инициализация, add, commit, status, log, diff, ветки (branch, checkout, merge)

Git: работа с удаленным репозиторием (push, pull, fetch)

Linux: базовые команды (pwd, ls, cd, mkdir, cp, mv, rm, cat, less, head, tail)

Linux: права доступа (chmod, chown)

IDE: установка, базовые настройки, отладка с breakpoints

Bash: переменные, if, for, while, простые функции

Middle Level (должен знать 100%)

Git: rebase vs merge, cherry-pick, stash, reset vs revert, reflog, разрешение конфликтов

Git: submodules, bisect

Linux: управление процессами (ps, top, kill), системные вызовы (strace), сеть (netstat, tcpdump)

Linux: systemd (создание сервисов), cron (планировщик)

IDE: удаленная разработка, условные breakpoints, профилирование

Bash: awk, sed, grep, find, xargs, регулярные выражения

Senior Level (должен знать 100%)

Git: внутреннее устройство (blob, tree, commit), Git Flow/Trunk-Based, Git LFS, hooks

Git: GPG подпись, защита веток, filter-branch, git gc

Linux: файловые системы (inode, ext4), виртуальная память (swap, mmap), cgroups/namespaces

Linux: SELinux/AppArmor, производительность (perf, ftrace), sysctl

IDE: кастомизация для команды, интеграция с CI/CD, анализ производительности IDE

Bash: безопасность скриптов, getopts, автоматизация деплоя, jq/yq

🦾 ПРАКТИЧЕСКИЙ ПРОЕКТ ДЛЯ ЗАКРЕПЛЕНИЯ ДОМЕНА 0

Задание: Создать систему для автоматического деплоя C++ приложения.

Шаги:

1. Написать C++ приложение "Hello World" с CMake
2. Создать Git-репозиторий, закоммитить код
3. Написать Bash-скрипт для сборки приложения (CMake + make)
4. Написать Bash-скрипт для создания Docker-образа (Dockerfile с multi-stage)
5. Написать Bash-скрипт для деплоя в Kubernetes (apply deployment.yaml)
6. Создать пайплайн в GitHub Actions, который автоматически собирает и деплоит при push в main
7. Настроить cron-задачу для еженедельного бэкапа базы данных